AFUSION · PARIS BAGUETTE PROGRAMME · PMO / QT34

Board Discipline Handbook

The 30 audit controls the PMO checks on our Azure DevOps board — split into what **every team member** owns day to day, and what the **Scrum Master** owns across the sprint.

11	19	30
MEMBER RULES	SCRUM MASTER RULES	CONTROLS IN TOTAL

Version 1.0 · 27 October 2026 · Source: PMO audit control list (QT34 §6.2–§6.3) · Board: devops.imt-soft.com/Inheritance/AFusion
Automated audit runs daily at 09:00; results are published to the PMO audit report and to the team's Element room. This handbook explains the rules — the audit checks them.

HOW THIS WORKS

Read this page once, then keep the checklist

The audit is automated. It reads the board every morning, scores each control as pass, warn or fail, and posts the gaps. Most gaps are not policy problems — they are three missing fields and one forgotten state change.

> **Why it matters beyond the audit.** The board is how we answer "how is the sprint going?" to the client. When descriptions are empty, effort is blank or a parent is still Active with every child Done, the burndown becomes fiction and the PMO cannot sign off the sprint. Every rule below exists to keep that picture honest.

The five habits that fix most of the board

1 · Describe before you start

Four lines — context, requested, expected output, out of scope. Then 2–5 acceptance criteria. Two minutes, and rules #05 and #06 are done for the life of the item.

2 · Estimate at planning, not later

Effort on the PBI, Original Estimate on the tasks. Anything blank at planning is a rule #15 gap and an untrustworthy sprint load.

3 · Move the state when reality moves

Start → Active. Merged and tested → Resolved. Evidence attached → Closed. Parents follow their children; never leave a parent behind.

4 · Touch it every two days

Remaining Work after stand-up, or one comment saying where it stands. An item silent for five days reads as abandoned.

5 · Leave a trail

`AB#id` in the commit and the PR, one reviewer approving, evidence in the comment before Close. That is rules #23, #24 and #25 in one habit.

And the Scrum Master's half

Sprint set-up, capacity, ownership, blockers, backlog order and the review/retro evidence — the 19 controls in the second half of this document.

INDEX

The 30 controls at a glance

Every control, who owns it, how often it is checked, and whether it applies always or only when the condition exists (a sprint is running, code is produced, an estimate applies).

#	CONTROL	OWNER	CADENCE	APPLICABILITY
01	Project workspace is available and current	Scrum Master	Monthly	Required
02	Project member list is current	Scrum Master	Monthly	Required
03	Access is aligned with roles and personnel changes	Scrum Master	Monthly	Required
04	Work requests are recorded in the backlog	Scrum Master	Weekly	Required
05	Work items contain sufficient detail	Member	Weekly	Required
06	Acceptance criteria are defined where required	Member	Bi-weekly	Conditional
07	Backlog priority / order reflects business priority	Scrum Master	Weekly	Required
08	Large work is decomposed when needed	Scrum Master	Bi-weekly	Conditional
09	Requirement changes are reflected in the backlog	Scrum Master	Weekly	Required
10	Current sprint / iteration is identified	Scrum Master	Bi-weekly	Conditional
11	Sprint goal / objective is understood	Scrum Master	Bi-weekly	Conditional
12	Team workload / capacity is considered	Scrum Master	Bi-weekly	Required
13	Planned work is selected / prioritized before execution	Scrum Master	Bi-weekly	Conditional
14	Work is broken into manageable tasks	Member	Bi-weekly	Conditional
15	Effort / points are recorded where estimation is applicable	Member	Weekly	Conditional
16	Active work items have a responsible owner	Scrum Master	Weekly	Required
17	Work-item status reflects actual progress	Member	Weekly	Required
18	Effort / remaining work is updated where used	Member	Weekly	Conditional
19	Active tasks / bugs / issues are updated regularly	Member	Weekly	Required
20	Bugs / issues are recorded and assigned when applicable	Member	Weekly	Conditional
21	Blockers / impediments are visible and followed up	Scrum Master	Weekly	Required
22	Progress can be monitored from project data	Scrum Master	Weekly	Required
23	Commits are linked to the related work item	Member	Weekly	Conditional
24	Pull Requests are linked and reviewed	Member	Weekly	Conditional
25	Completed work meets the Definition of Done	Member	Bi-weekly	Required
26	Sample traceability check passes end-to-end	Scrum Master	Monthly	Review Control
27	Sprint Review / demo is performed	Scrum Master	Bi-weekly	Conditional
28	Feedback is captured and reflected in follow-up work	Scrum Master	Bi-weekly	Required
29	Backlog is reviewed / reprioritized regularly	Scrum Master	Weekly	Required
30	Retrospective / improvement actions are tracked	Scrum Master	Bi-weekly	Conditional

PART ONE · 11 RULES

For every team member

What you own on the work items you touch — developers, QA, BA and anyone writing a PBI. These are the rules the audit most often fails, and the cheapest ones to fix.

#05 Work items contain sufficient detail

REQUIRED WEEKLY

2. Backlog / Requirements · QT34

What we check. Each item clearly states the requested work and the expected output.

How to satisfy it. Write the Description before you move the item to Active. A title alone is never enough. Cover four things: **Context** (where in the system), **Requested** (what to build or change), **Expected output** (what the user sees when it works), **Out of scope** (what not to touch).

> EXAMPLE
> Context: Supplier invoice import screen (AP module).
> Requested: accept PDF invoices from MoMo and parse the total.
> Expected output: imported invoice shows correct vendor, date and total; unmatched ones land in the review queue.
> Out of scope: OCR tuning for scanned images.

#06 Acceptance criteria are defined where required

CONDITIONAL BI-WEEKLY

2. Backlog / Requirements · QT34

What we check. Acceptance conditions are clear at the appropriate work-item / task level.

How to satisfy it. Every PBI and Bug carries 2–5 testable conditions in the **Acceptance Criteria** field. Write them as Given / When / Then, or as a tick-list a tester could run without asking you anything. Write them *before* the item enters the sprint, not after the code is done.

> EXAMPLE
> Given an invoice PDF with 3 line items
> When the importer runs
> Then 3 rows are created and the total matches to 0.01.

#14 Work is broken into manageable tasks

CONDITIONAL BI-WEEKLY

3. Sprint / Work Planning · QT34

What we check. Tasks are small enough to track and complete effectively.

How to satisfy it. Every PBI in the sprint has at least one **Task** child. Name tasks with a verb (Implement, Verify, Review, Deploy) and keep each one to roughly a day or less. If a task would take longer than two days, split it — the burndown depends on it.

> EXAMPLE
> Implement parser → Verify with 5 real PDFs → Code review → Deploy to UAT.

#15 Effort / points are recorded where estimation is applicable

CONDITIONAL WEEKLY

3. Sprint / Work Planning · QT34

What we check. Applicable work has an agreed estimate before or during planning.

How to satisfy it. Set **Effort** (story points or hours) on every PBI and Bug committed to a sprint, and **Original Estimate** on its tasks. Blank effort means nobody can tell whether the sprint is over- or under-loaded, and the burndown cannot be trusted. Estimate at planning — do not leave it for later.

> EXAMPLE
> PBI Effort = 5. Tasks: 4h / 2h / 2h / 4h.

#17 Work-item status reflects actual progress

REQUIRED WEEKLY

4. Sprint Execution & Tracking · QT34

What we check. State is current and matches the real condition of the work.

How to satisfy it. Move the item the moment reality changes: **New** → **Active** when you start, **Active** → **Resolved** when code is merged and tested, **Resolved** → **Closed** once the DoD evidence is attached. A parent (Feature/Epic) with all children Done but still sitting in New/Active is the single most common audit failure on this board.

> EXAMPLE
> All 4 tasks Done + PR merged + tested on UAT → the PBI goes to Resolved, not left Active.

#18 Effort / remaining work is updated where used

CONDITIONAL WEEKLY

4. Sprint Execution & Tracking · QT34

What we check. Tracking fields are current enough to support monitoring and reporting.

How to satisfy it. Every task in **In Progress** has a **Remaining Work** value, updated once a day right after stand-up. If you have not started it, Remaining = Original Estimate. When it is done, set Remaining to 0 and move it to Done — do not just leave the number frozen at yesterday's value.

> EXAMPLE
> Task 8h estimate → day 1 remaining 6h → day 2 remaining 2h → done, remaining 0.

#19 Active tasks / bugs / issues are updated regularly

REQUIRED WEEKLY

4. Sprint Execution & Tracking · QT34

What we check. Recent updates provide a reliable view of current work and blockers.

How to satisfy it. Anything you are working on gets touched at least every two working days: a state change, a Remaining Work update, or a short comment on where it stands. An item with no activity for five days reads as abandoned — to the board, to the PMO, and to whoever is planning next week.

> EXAMPLE
> Comment: “Parser handles 4/5 sample PDFs; MoMo multi-page layout still failing — 2h left.”

#20 Bugs / issues are recorded and assigned when applicable

CONDITIONAL WEEKLY

4. Sprint Execution & Tracking · QT34

What we check. Relevant bugs have an owner, a classification and the needed linkage.

How to satisfy it. Every defect that reaches the team becomes a **Bug** work item — not a chat message. It needs repro steps, Severity, an assignee, and a link to the PBI or Feature it affects. Fixing something quietly and never filing it leaves the quality history empty.

> EXAMPLE
> Bug: “Invoice total rounds down on 3-decimal currencies.” Repro: import sample_07.pdf. Severity 2. Link: parent PBI #58439.

#23 Commits are linked to the related work item

CONDITIONAL WEEKLY

5. Quality & Traceability · QT34

What we check. Sampled code changes can be traced to the correct task or work item.

How to satisfy it. Put `AB#<id>` in the commit message (or link the commit from the work item's Development section). One commit, one item — do not squash five PBIs into a single “fix stuff” commit with no reference. Without this link, nobody can answer “what changed for this requirement?”.

> EXAMPLE
> git commit -m "Parse MoMo multi-page invoices AB#58439"

#24 Pull Requests are linked and reviewed

CONDITIONAL WEEKLY

5. Quality & Traceability · QT34

What we check. Sampled PRs link to work items and show completed code review before merge.

How to satisfy it. PR title or description carries `AB#<id>`; at least one reviewer approves before merge; the reviewer is not the author. Review comments stay on the PR, not in chat. An unreviewed merge on a completed PBI is a traceability gap even if the code is fine.

> EXAMPLE
> PR “Invoice importer: multi-page support AB#58439” → reviewer approves → merge → PBI moves to Resolved.

#25 Completed work meets the Definition of Done

REQUIRED BI-WEEKLY

5. Quality & Traceability · QT34

What we check. Done items carry sufficient evidence that acceptance conditions were met.

How to satisfy it. Before you close anything, attach evidence in a comment: the test result, a screenshot, a log line, or the UAT sign-off — and tick the acceptance criteria you actually verified. “It works on my machine” is not evidence. If a criterion was not verified, the item is Resolved, not Closed.

> EXAMPLE
> Comment: “Verified on UAT build 1.4.2 — 5/5 sample PDFs imported, totals match. Screenshot attached.”

PART TWO · 19 RULES

For the Scrum Master

Sprint set-up, planning evidence, tracking integrity and the improvement loop. Most of these need a written artefact in the wiki — the PMO checks evidence, not intention.

#01 Project workspace is available and current

REQUIRED MONTHLY

1. Project & Team · QT34

What we check. A current workspace exists in the approved tool and is used for tracking.

How to satisfy it. Keep every active workstream inside the AFusion ADO project: areas, iterations and boards current. No parallel tracking in spreadsheets or chat — if it is not in ADO, the PMO cannot see it and it did not happen.

#02 Project member list is current

REQUIRED MONTHLY

1. Project & Team · QT34

What we check. The list holds name, role, join date, leave date and notes.

How to satisfy it. Update the Project Members wiki page whenever someone joins or leaves — same week, not at sprint close. The member list is the reference the PMO uses to check access and capacity, so a stale list fails three controls at once.

> EXAMPLE
> Wiki: /AFusion-Rpa.wiki/844/Project-Members

#03 Access is aligned with roles and personnel changes

REQUIRED MONTHLY

1. Project & Team · QT34

What we check. Access is granted by role and removed when membership changes.

How to satisfy it. Grant by role (Contributor for the team, Stakeholder for observers), and remove access the day someone leaves the project. Review the team page monthly against the member list.

#04 Work requests are recorded in the backlog

REQUIRED WEEKLY

2. Backlog / Requirements · QT34

What we check. All current known work is visible in ADO.

How to satisfy it. Anything the team is expected to do exists as a work item — including small requests, hotfixes and “can you just” asks. This is the control the PMO cannot verify from data alone, so it needs your confirmation each week: no work off-board.

#07 Backlog priority / order reflects business priority

REQUIRED WEEKLY

2. Backlog / Requirements · QT34

What we check. The next work to perform is identifiable from backlog order.

How to satisfy it. Re-rank the top of the backlog weekly with the PO. Order is the message: item 1 is what the team starts next. Never insert above the top of a sprint already in flight, and never let two items share a rank.

#08 Large work is decomposed when needed

CONDITIONAL BI-WEEKLY

2. Backlog / Requirements · QT34

What we check. Large requirements are broken into manageable items.

How to satisfy it. Epics → Features → PBIs before the work enters a sprint, so each PBI is a few days at most. A 40-point PBI that sits in the sprint for three weeks is a planning failure, not a tracking one.

#09 Requirement changes are reflected in the backlog

REQUIRED WEEKLY

2. Backlog / Requirements · QT34

What we check. Material scope, output or priority changes are recorded in the item.

How to satisfy it. Any change after sprint start — new field, changed output, dropped scope — is recorded the same day: update the item's Description/AC and add a comment saying what changed and why. Scope that moves without a record makes the burndown lie.

> EXAMPLE
> Comment: “2026-10-24 — client added multi-currency totals. Scope +3 pts, agreed with PO.”

#10 Current sprint / iteration is identified

CONDITIONAL BI-WEEKLY

3. Sprint / Work Planning · QT34

What we check. The current work period is visible and active items are in it.

How to satisfy it. Iteration dates set before the sprint starts, all committed items in that iteration path, and the previous iteration closed out (nothing left Active in a finished sprint).

#11 Sprint goal / objective is understood

CONDITIONAL BI-WEEKLY

3. Sprint / Work Planning · QT34

What we check. A clear sprint objective is recorded or understood by the team.

How to satisfy it. One sentence, written in the sprint planning notes and repeated at stand-up day one. If the team cannot say it from memory on day two, it was not a goal.

#12 Team workload / capacity is considered

REQUIRED BI-WEEKLY

3. Sprint / Work Planning · QT34

What we check. Availability, leave, training and current load are considered.

How to satisfy it. Enter each member's capacity and days off in ADO before planning, then check committed effort against it. Note in the planning minutes that capacity was reviewed — the PMO needs the evidence, not just the fact.

#13 Planned work is selected / prioritized before execution

CONDITIONAL BI-WEEKLY

3. Sprint / Work Planning · QT34

What we check. The sprint backlog reflects agreed priorities.

How to satisfy it. Sprint backlog is agreed and ordered before anyone starts coding; mid-sprint additions go through the same ordering, not straight to the top.

#16 Active work items have a responsible owner

REQUIRED WEEKLY

4. Sprint Execution & Tracking · QT34

What we check. Each active item has an accountable assignee.

How to satisfy it. No item in Active without a name against it. Unassigned work is invisible work — it cannot be chased, estimated or closed. If the owner is genuinely unknown, the item stays in New until it is decided.

#21 Blockers / impediments are visible and followed up

REQUIRED WEEKLY

4. Sprint Execution & Tracking · QT34

What we check. Material blockers have an owner, an action and a current status.

How to satisfy it. When someone is blocked, it becomes visible the same day: a Bug/Issue with the `blocked` tag, or an Impediment entry, with an owner and a next action. Review blockers daily at stand-up and record the resolution.

#22 Progress can be monitored from project data

REQUIRED WEEKLY

4. Sprint Execution & Tracking · QT34

What we check. Board, query and reports give a reliable view of workload and progress.

How to satisfy it. This is the summary control: it passes when statuses, remaining work and ownership are honest. Check the sprint burndown weekly — if the line looks impossible, the data is wrong, not the team.

#26 Sample traceability check passes end-to-end

REVIEW CONTROL MONTHLY

5. Quality & Traceability · QT34

What we check. 2–3 completed items trace from request to execution, review and Done.

How to satisfy it. Monthly, pick 2–3 closed items and walk the chain: request → PBI → tasks → commits/PRs → test evidence → Done. Fix the breaks you find; they are almost always the same three (unlinked commit, no AC, no evidence).

#27 Sprint Review / demo is performed

CONDITIONAL BI-WEEKLY

6. Sprint Review & Improvement · QT34

What we check. Completed work is reviewed with the relevant stakeholders.

How to satisfy it. Hold the review at sprint end, demo against the acceptance criteria (not slides), and record attendees, what was accepted and what was rejected in the wiki.

#28 Feedback is captured and reflected in follow-up work

REQUIRED BI-WEEKLY

6. Sprint Review & Improvement · QT34

What we check. Feedback is recorded and creates or updates work items.

How to satisfy it. Every piece of feedback from the review becomes a work item or an update to an existing one, with the requester named. Feedback that lives only in meeting notes is lost by the next sprint.

#29 Backlog is reviewed / reprioritized regularly

REQUIRED WEEKLY

6. Sprint Review & Improvement · QT34

What we check. Backlog order stays aligned with the latest business priority.

How to satisfy it. Weekly 30-minute backlog review with the PO: re-order, split, drop, and record the date of the review. Note it in the sprint review page so the check has evidence.

#30 Retrospective / improvement actions are tracked

CONDITIONAL BI-WEEKLY

6. Sprint Review & Improvement · QT34

What we check. Improvement actions have an owner and a follow-up status.

How to satisfy it. Each retro produces 1–3 actions, each with a name and a date, tracked in the wiki retro page and reviewed at the next retro. An action without an owner is a wish.

> **If the audit flags one of your items.** Three steps, same day — **1.** open the link in the report: it names the item and the exact field; **2.** fix the field itself (description, AC, effort, state, owner, link), not the symptom; **3.** add a one-line comment saying what you changed, so the next audit and the PMO can see it was handled. If the finding is wrong, say so in the comment — the audit is a checklist, not a verdict.

REFERENCE

Field map and who to ask

Where each audited field lives on an ADO work item — and who to go to when a rule does not make sense for your work.

Where the audited data lives

FIELD	RULES	WHAT GOOD LOOKS LIKE
Title	05, 20	A verb and an object — “Parse MoMo multi-page invoices”, not “invoices”.
Description	05, 09	Context / Requested / Expected output / Out of scope. Never empty.
Acceptance Criteria	06, 25	2–5 testable conditions; ticked as they are verified.
State	17, 19	Matches reality today, including parents.
Assigned To	16, 20	A named owner on everything Active.
Effort / Story Points	15	Set at planning; never blank on committed work.
Original Estimate	15, 18	On tasks, at creation.
Remaining Work	18, 22	Updated daily after stand-up; 0 when done.
Iteration	10, 13	Correct sprint path; no work left in a closed sprint.
Area	01, 22	Right module, so reports and filters work.
Priority / backlog order	07, 29	One clear next item at the top.
Links (parent, commits, PRs)	08, 23, 24, 26	Parent set, `AB#id` in commits and PRs.
Tags	21	`blocked`, `scope-added` where applicable.
Comments	09, 19, 25	Where the work stands, and the evidence before Close.

Who to ask

IF IT IS ABOUT…	ASK	BECAUSE
Sprint set-up, capacity, iteration paths, board structure	**Scrum Master**	They configure the sprint and hold the planning evidence.
Priority, scope, “is this in the sprint?”	**Product Owner**	Order on the backlog is the answer; chat is not.
How a control is interpreted, or why it failed	**PMO**	The control text and the evidence rule are theirs (QT34).
What the automated fix changed on your item	**The audit report**	Every auto-corrected item carries a `[Board Hygiene]` comment with the reason.

> **What the automation does and does not do.** A remediation job runs after each audit and fills the safe gaps on its own — drafting empty descriptions, deriving effort from child tasks, appending backlog priority, tagging scope changes — and comments on every item it touches. It will never assign an owner, change a state, close a parent or rewrite text a human wrote: those stay with you, and they are listed in the report under *Needs a human*.

RHYTHM

The weekly rhythm

Ten minutes a day, thirty a week, plus the sprint bookends. None of it is extra work — it is the same work, recorded where the audit can see it.

Daily · 10 minutes

- Move items whose reality changed
- Update Remaining Work on In Progress tasks
- Raise blockers so they are visible
- Comment on anything silent for two days

Weekly · 30 minutes (SM)

- Re-rank the backlog with the PO
- Review capacity and sprint burndown
- Chase unassigned and stale Active items
- Record the backlog review date in the wiki

Sprint start (SM)

- Iteration dates and capacity entered
- Sprint goal written in the planning notes
- Backlog ordered before execution starts
- Every PBI estimated and broken into tasks

Sprint end (SM)

- Review held, feedback captured as work items
- Traceability spot-check on 2–3 done items
- Retro actions with owners recorded
- Nothing left Active in the closed sprint

AFusion · Board Discipline Handbook · v1.0 · 27 October 2026 — derived from the PMO audit control list (QT34 §6.2–§6.3). Re-read when the control list changes; the audit itself always reflects the live list.